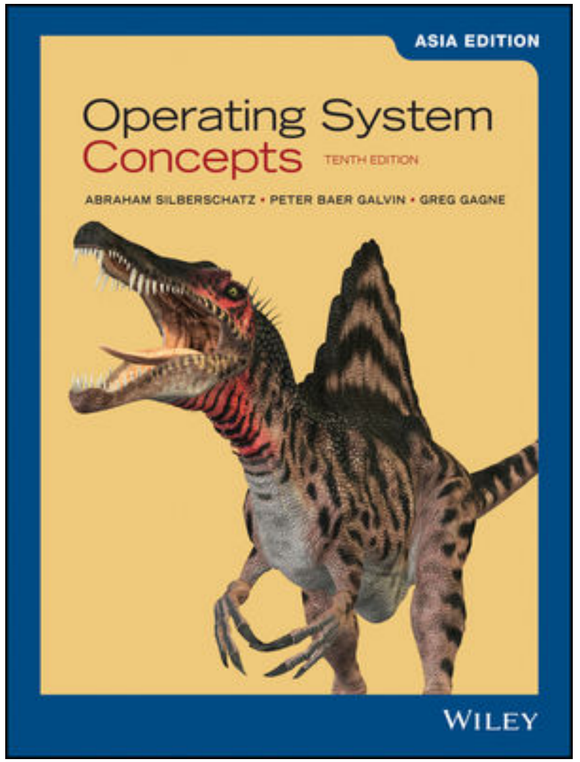
Chapter 5
CPU Scheduling

Chih-Yu Lin
National Taiwan Ocean University

Updated date: 2025/7/13

Textbook

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts, Asia Edition,” 10/e, ISBN: 978-1-119-58616-6, Wiley



Outline

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU Scheduling
- Algorithm Evaluation

Outline

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU Scheduling
- Algorithm Evaluation

Basic Concepts

- Maximum CPU utilization obtained with multiprogramming

- CPU - I/O Burst Cycle
- CPU Scheduler 排程器，决定下一个 Slice 换谁
- Preemptive and Nonpreemptive Scheduling 会考
- Dispatcher

强制性 Time sharing　非强制 Time sharing

少见 (e.g. Windows 3.1)

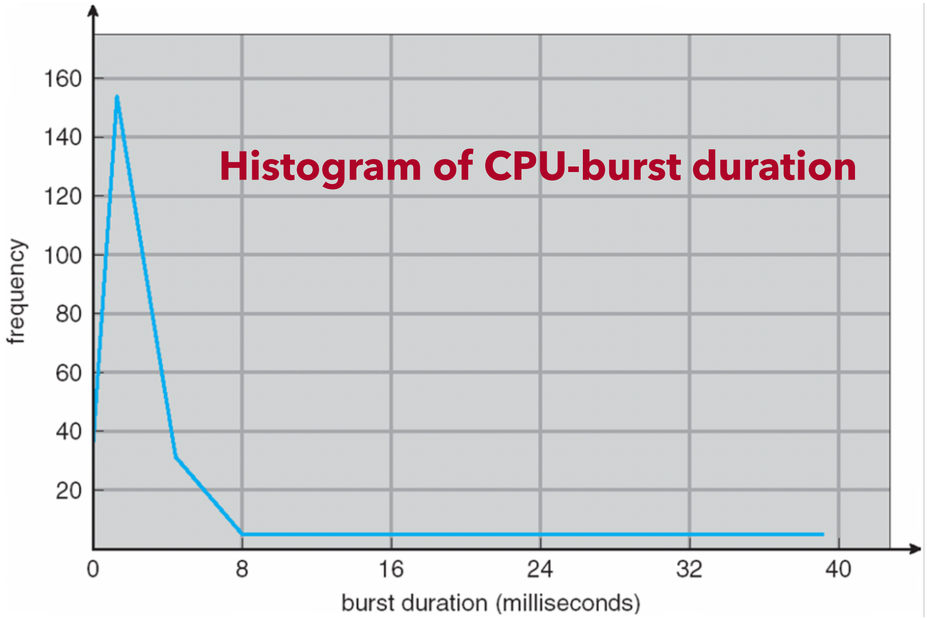
(似 Context Switch)

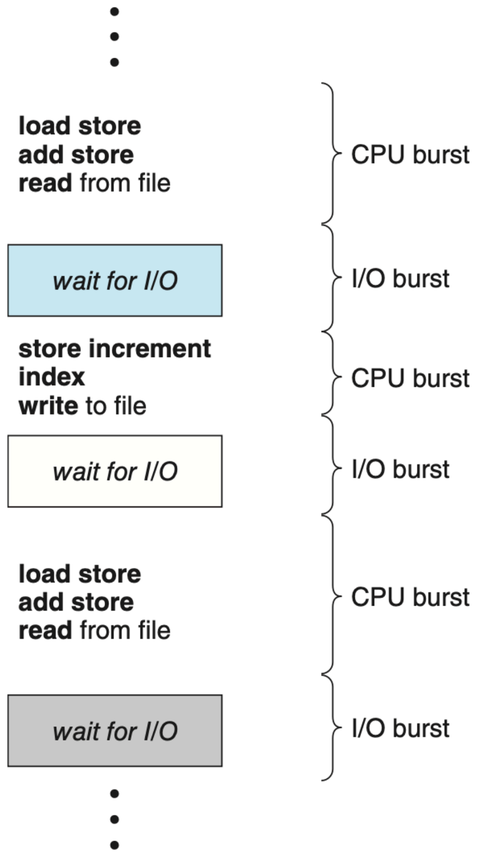
Midterm Time Sharing 解释必考

CPU - I/O Burst Cycle

程式執行过程中，CPU 執行 Time (CPU Brust) 和 I/O 等待時間 (I/O Brust) 的週期性循環

- CPU–I/O Burst Cycle – Process execution consists of a cycle of CPU execution and I/O wait
- CPU burst followed by I/O burst
- CPU burst distribution is of main concern





CPU Scheduler

- CPU Scheduler selects a process from the processes in memory that are ready to execute and allocates the CPU to that process
- Ready Queue can be implemented as
  - a FIFO queue
  - a priority queue
  - a tree
  - imply an unordered linked list
- The records in the queues are generally process control blocks (PCBs) of the processes.

Chapter. 3 / P12

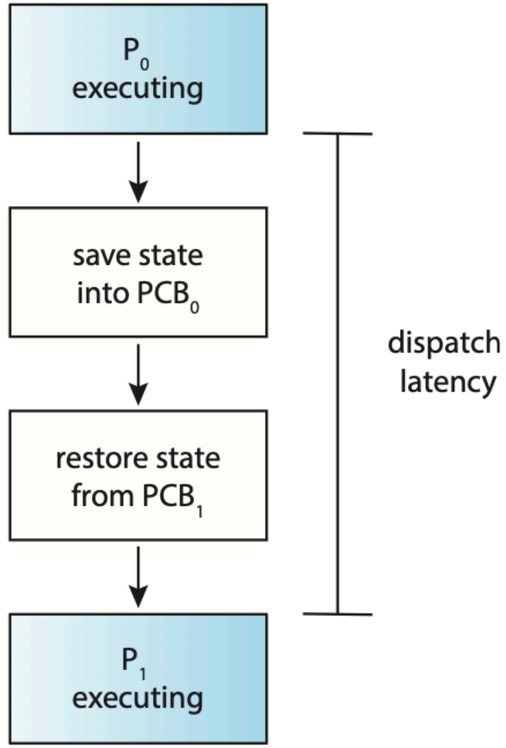
Preemptive and Nonpreemptive Scheduling

- CPU-scheduling decisions may take place under the following four circumstances
  - Switches from running to waiting state
  - Switches from running to ready state (e.g, interrupt / context switch)
  - Switches from waiting to ready (e.g. I/O 完，発送 Interrupt 給CPU)
  - Terminates
- When scheduling takes place only under circumstances 1 and 4, we say that the scheduling scheme is **nonpreemptive** or **cooperative**. Otherwise, it is **preemptive**.
  非搶佔式 (不会強制 Time Sharing)
- All modern operating systems including Windows, macOS, Linux, and UNIX use **preemptive** scheduling algorithms.

Dispatcher

把CPU的使用、控制權交給另一个Process

- Another component involved in the CPU-scheduling function is the dispatcher
- The dispatcher is the module that gives control of the CPU's core to the process selected by the CPU scheduler; this involves:
  - Switching context
  - Switching to user mode
  - Jumping to the proper location in the user program to restart that program
- **Dispatch latency** – time it takes for the dispatcher to stop one process and start another running

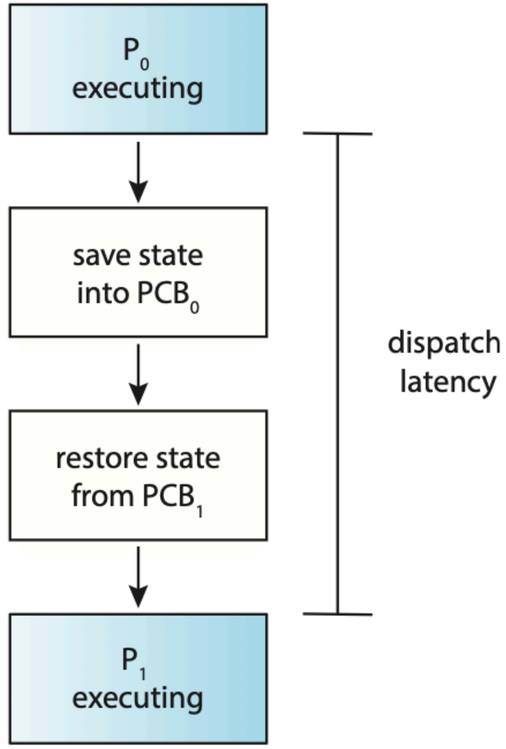


Dispatcher

Item	Description
Context Switch	The **action or process** of switching the CPU from one process to another.
Dispatcher	The **module or entity** that performs the context switch and handles the necessary steps to start executing the selected process.

✅ **Simple analogy**:

- **Context Switch** = The **act of switching**.
- **Dispatcher** = The **component that carries out the switching process**.



Outline

- Basic Concepts
- Scheduling Criteria (衡量/評判 Algorithms 的標準)
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU Scheduling
- Algorithm Evaluation

Scheduling Criteria

- (Max) **CPU utilization** – keep the CPU as busy as possible → 讓CPU越忙越好
- (Max) **Throughput** – # of processes that complete their execution per time unit → 最大化同樣時間內完成的 Processes 數
- (Min) **Turnaround time** – amount of time to execute a particular process → 一个Process 從進 Ready Queue 到執行完的時間
- (Min) **Waiting time** – amount of time a process has been waiting in the ready queue → 在 ready Queue 中等待的時間
- (Min) **Response time** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment) → 響应時間：從使用者/Program 発出一个请求，到系統开始回応这个请求的所需時間

現今个人电腦比較強调这兩个

Outline

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU Scheduling
- Operating-System Examples
- Algorithm Evaluation

Scheduling Algorithms

重点、必考!

- FCFS (First-Come, First-Served)
- Shortest-Job-First Scheduling

→必考这两个!

- Round-Robin Scheduling
- Priority Scheduling
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

Scheduling Algorithms

- **FCFS (First-Come, First-Served)**
- Shortest-Job-First Scheduling
- Round-Robin Scheduling
- Priority Scheduling
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

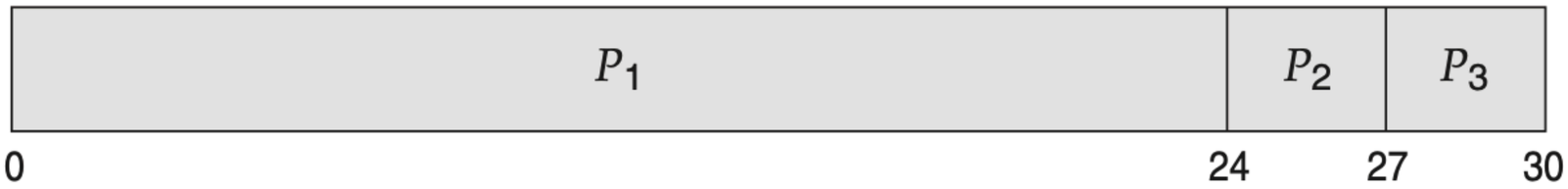
First-Come, First-Served Scheduling

谁先来、谁先用 (ignore Time-Sharing)

- Suppose that the processes arrive in the order: P1 , P2 , P3

Process	Burst Time (使用CPU的時間)
P_1	24
P_2	3
P_3	3

- The **Gantt Chart** for the schedule is:

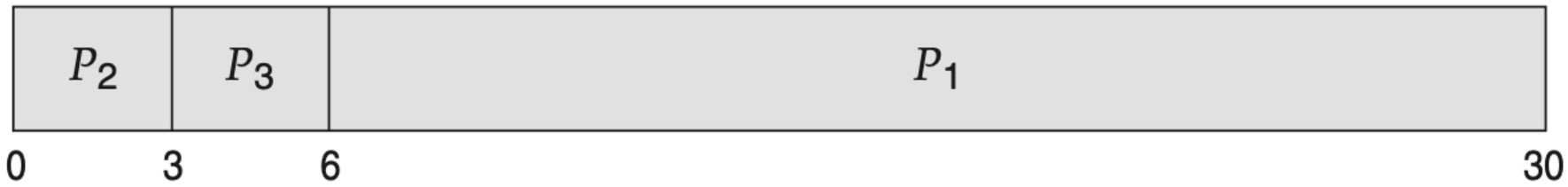


  - Waiting time for P1 = 0; P2 = 24; P3 = 27
  - Average waiting time: (0 + 24 + 27)/3 = 17

First-Come, First-Served Scheduling

- Suppose that the processes arrive in the order: P2 , P3 , P1

  The **Gantt Chart** for the schedule is:



  - Waiting time for P1 = 6; P2 = 0; P3 = 3
  - Average waiting time: (6 + 0 + 3)/3 = 3

- **Convoy effect** - short process behind long process

  護送效応

  - Consider one CPU-bound and many I/O-bound processes

Convoy effect

- A resource-intensive process blocks other processes, causing them to line up waiting, similar to a convoy of vehicles on a highway
- The Convoy Effect typically occurs in the following situations:
    1. Short tasks blocked by long tasks: When a process requiring long execution time obtains the processor or resource, many short tasks that could complete quickly must wait.
    2. Non-preemptive scheduling: In non-preemptive scheduling systems, once a process begins execution, it continues until completion, blocking, or voluntarily releasing resources.

Scheduling Algorithms

- FCFS (First-Come, First-Served)
- **Shortest-Job-First Scheduling**
- Round-Robin Scheduling
- Priority Scheduling
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

Shortest-Job-First (SJF) Scheduling

誰時間短先做

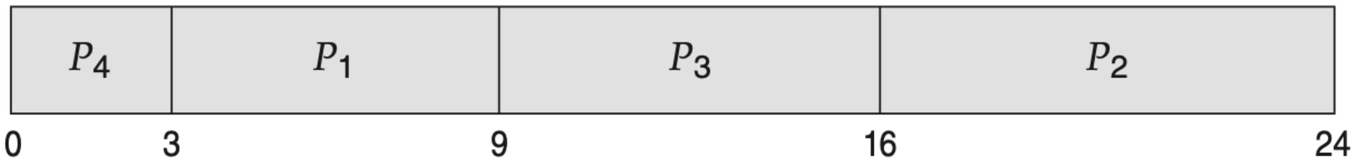
- Associate with each process the length of its next CPU burst
  - Use these lengths to schedule the process with the shortest time

- Example
  - Average waiting time = (3 + 16 + 9 + 0) / 4 = 7

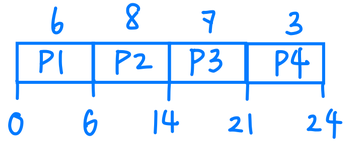
(Assume arrival at the same time)

Process	Burst Time	
P_1	6	2
P_2	8	4
P_3	7	3
P_4	3	1

waiting Time 0 3 9 16



FCFS:



(0+6+14+21) / 4 = 10.25

Shortest-Job-First (SJF) Scheduling

SJF 是个很理想的算法

- SJF is optimal – gives minimum average waiting time for a given set of processes
  - The difficulty is knowing the length of the next CPU request (CPU 沒辦法知道 Process 的 Brust Time)
  - Predicted based on the previous CPU bursts (由先前的使用情況來預測)

預測 Process 使用時間的算法：指數平滑法

- Exponential Average (exponential smoothing formula)
  - t_n - actual length of n^{th} CPU burst
  - τ_{n+1} - predicted value for the next CPU burst
  - $\tau_{n+1} = \alpha t_n + (1-\alpha)\tau_n$, where $0 \le \alpha \le 1$ (平滑因子)

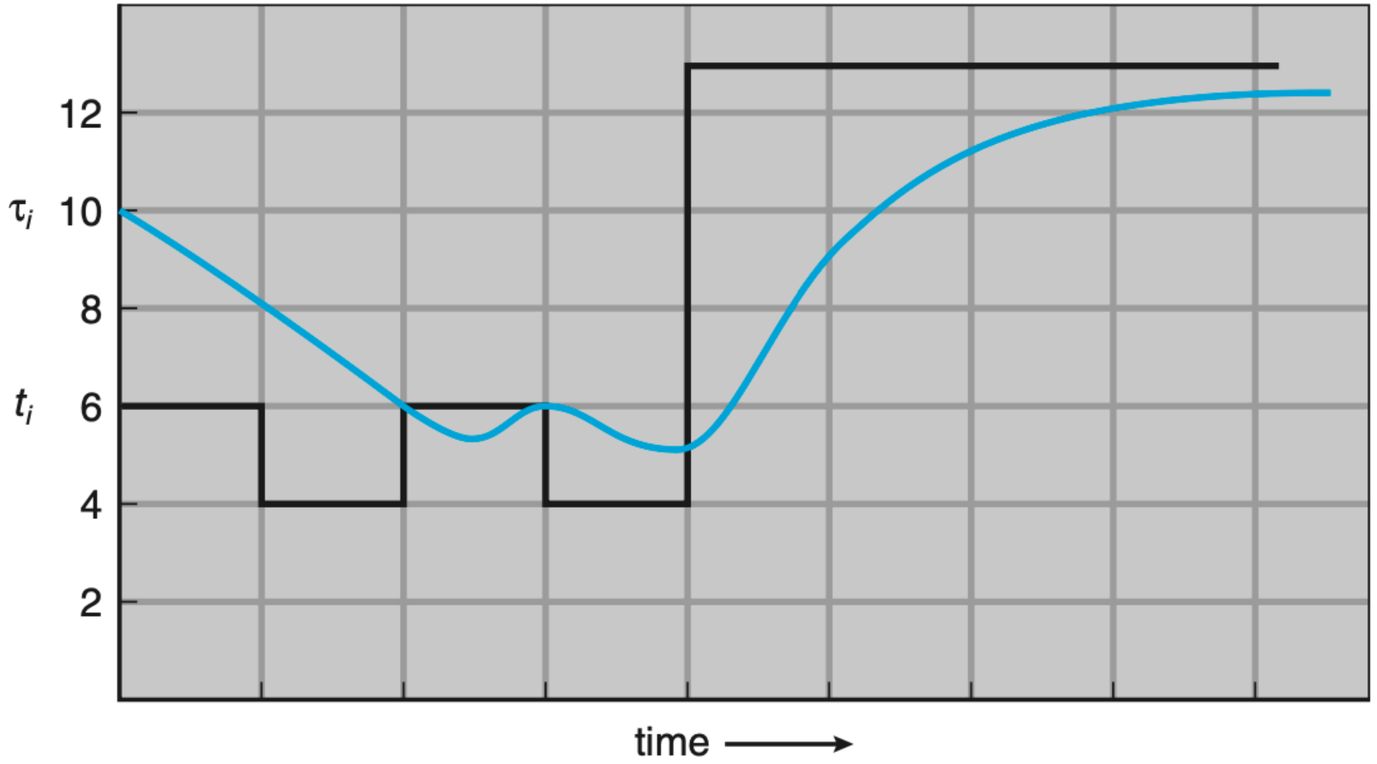
t_n: now Process 的實際執行時間

τ_n: 拿這項的預測時間去預測下一項時間

$$\tau_{n+1} = \alpha t_n + \alpha(1-\alpha)t_{n-1} + \alpha(1-\alpha)^2 t_{n-2} + \cdots$$

Given by ChatGPT

Exponential Average



$\alpha = 1/2$

CPU burst (t_i)		6	4	6	4	13	13	13	...
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

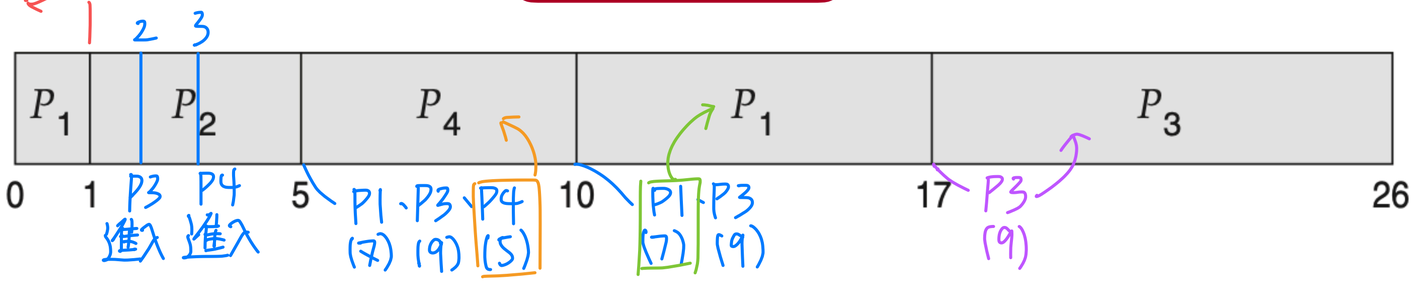
Shortest-Job-First (SJF) Scheduling

- The SJF algorithm can be either preemptive or nonpreemptive.
- Preemptive SJF scheduling is sometimes called shortest-remaining- time-first scheduling
- Example

Process	Arrival Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

NP ① ② ④ ③

preemptive 的 SFJ
改比"剩下要執行的時間"
剩最短的優先次



Preemptive:
P1: 10-1=9 中間等了9s
P2: 1-1=0 直接開始做
P3: 17-2=15
P4: 5-3=2

P2進入比較 (4 & 7比較!)
P1已經做1s了

FCFS:
P1 P4 P1 P3
0 8 12 21 26
0+(8-1)+(12-2)+(21-3)
=7+10+18=8.75秒

- Preemptive Version: [(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5 milliseconds
- Nonpreemptive Version:[0+(8-1)+(12-2)+(17-3)]/4 = 31/4 = 7.75 milliseconds

P1 P2 P4 P3
0 8 12 17 26

↘ 開始時間-Arrival Time!

Scheduling Algorithms

- FCFS (First-Come, First-Served)
- Shortest-Job-First Scheduling
- **Round-Robin Scheduling**
- Priority Scheduling
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

Round-Robin Scheduling

研究所常考！/ 概念類似 Time Sharing / 也是 FCFS，但是是 Preemptive，AVOID 護送效应

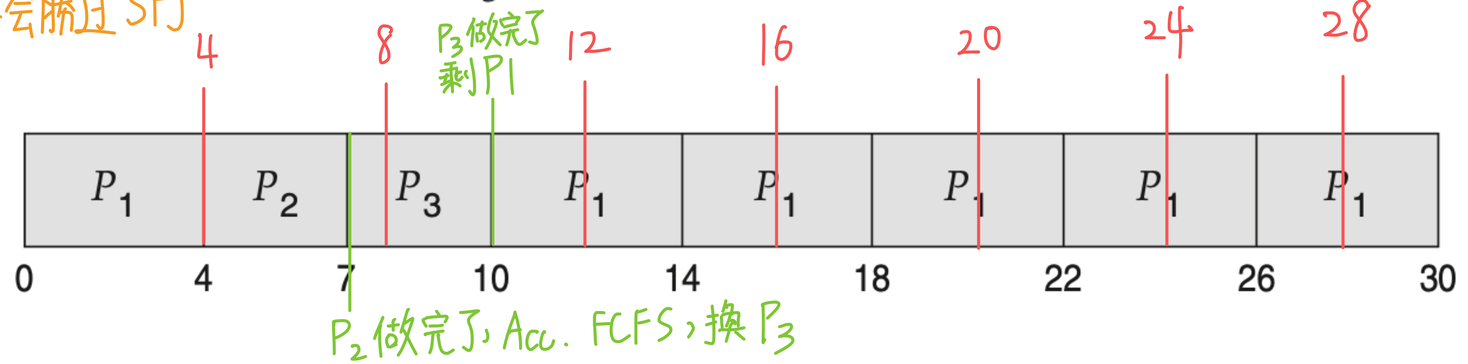
- The round-robin (RR) scheduling algorithm is similar to FCFS scheduling, but **preemption** is added to enable the system to switch between processes.
- **time quantum / time slice (q)** ⇒ 規定每个 task 經过 q 時間後就一定得換人
  - a fixed time interval allocated to each process in the ready queue
  - usually 10-100 milliseconds
- Example (q=4)

*FCFS

Process	Burst Time
P_1	24
P_2	3
P_3	3

Waiting Time:
P1: 10 - 4 = 6 (只有中間等6s)
P2: 4
P3: 7
→ $\frac{(6+4+7)}{3} = 5.6$

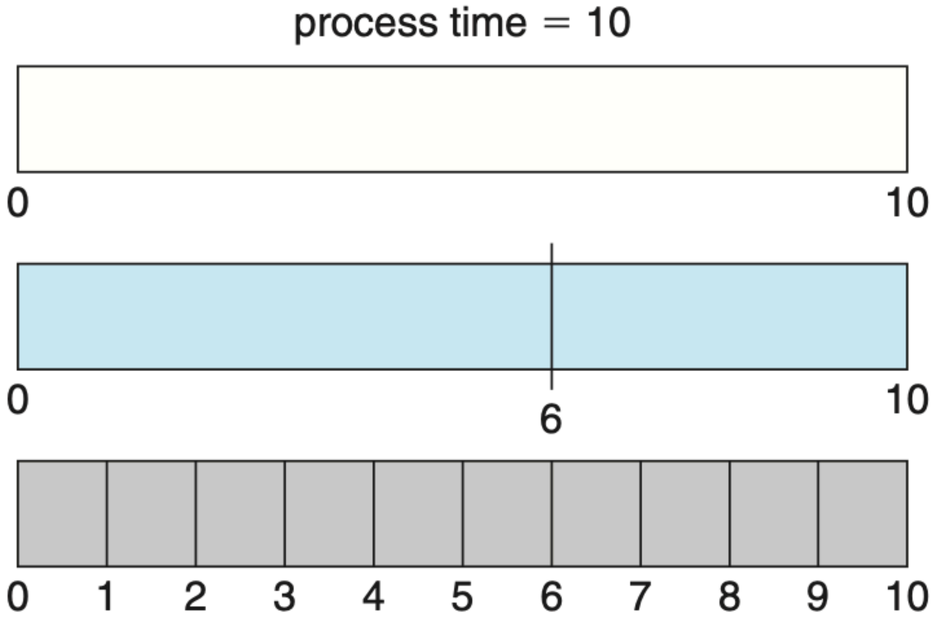
Round-Robin 確保每个 Process 執行的公平性
但其 Waiting Time 不見得会勝过 SFJ



Round-Robin Scheduling

- Performance
  - large q ⇒ FCFS → 如果q太大，RR就会变成FCFS
  - small q ⇒ q must be large with respect to context switch, otherwise overhead is too high

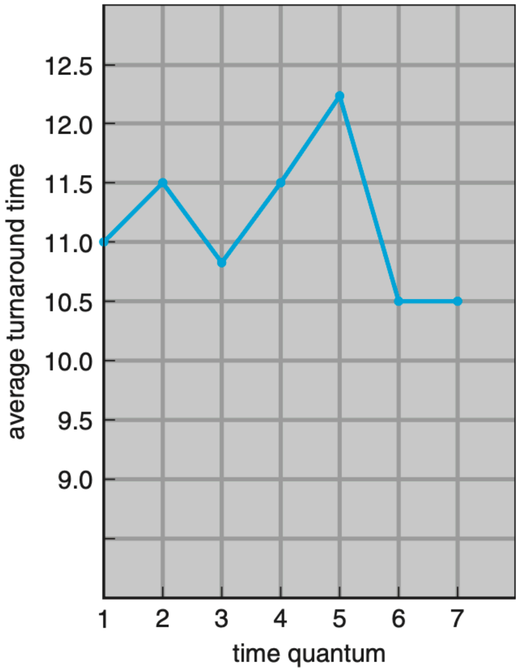
← 如果q太小，会发生太多次 context switch，時間开銷大



quantum	context switches
12	0
6	1
1	9

Round-Robin Scheduling

- Performance
  - Turnaround time – amount of time to execute a particular process
    - also depends on the size of the time quantum
  - In general, the average turnaround time can be improved if most processes finish their next CPU burst in a single time quantum
  - **80% of CPU bursts should be shorter than q**



process	time
P_1	6
P_2	3
P_3	1
P_4	7

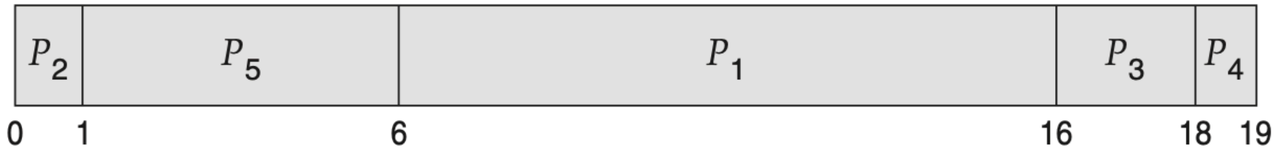
Scheduling Algorithms

- FCFS (First-Come, First-Served)
- Shortest-Job-First Scheduling
- Round-Robin Scheduling
- **Priority Scheduling**
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer ⇒ highest priority)
  - Equal-priority processes are scheduled in FCFS order
  - SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Example

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2



The average waiting time is 8.2 milliseconds.

Priority Scheduling

- Priority scheduling can be either preemptive or nonpreemptive
  - When a process arrives at the ready queue, its priority is compared with the priority of the currently running process
    - **Preemptive**: preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process
    - **Nonpreemptive**: simply put the new process at the head of the ready queue
- Problem:
  - **starvation** (**indefinite blocking**): low priority processes may never execute
  - Solution: **Aging** – as time progresses increase the priority of the process

Process 饿死：一个Process的优先度太低，所以一直被插队，用不到CPU

↖ 随着Process等待时间越久，其Priority会逐渐提高

Scheduling Algorithms

- FCFS (First-Come, First-Served)
- Shortest-Job-First Scheduling
- Round-Robin Scheduling
- Priority Scheduling
- **Multilevel Queue Scheduling**
- Multilevel Feedback Queue Scheduling

Multilevel Queue Scheduling

- Ready queue is partitioned into separate queues
  - foreground (interactive)
  - background (batch)

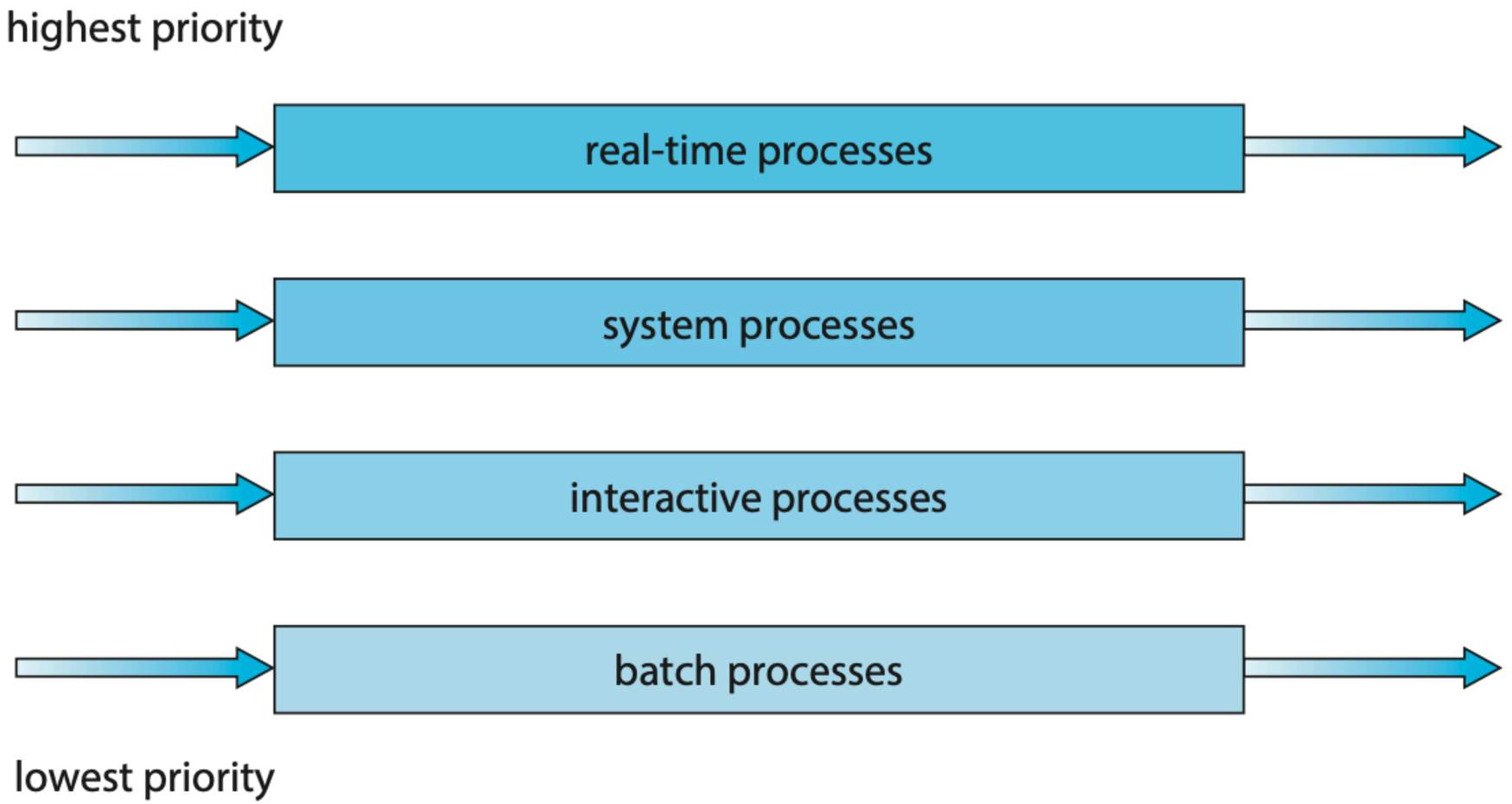
互动UI、keyboard、Mouse Input……

Multilevel Queue 將所有的task都区分成較重要和不重要的Queue 各自維護。

後臺下載、更新……

- Each queue has its own scheduling algorithm
  - foreground – RR
  - background – FCFS
- Scheduling must be done between the queues
  - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
  - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR, 20% to background in FCFS

Multilevel Queue Scheduling

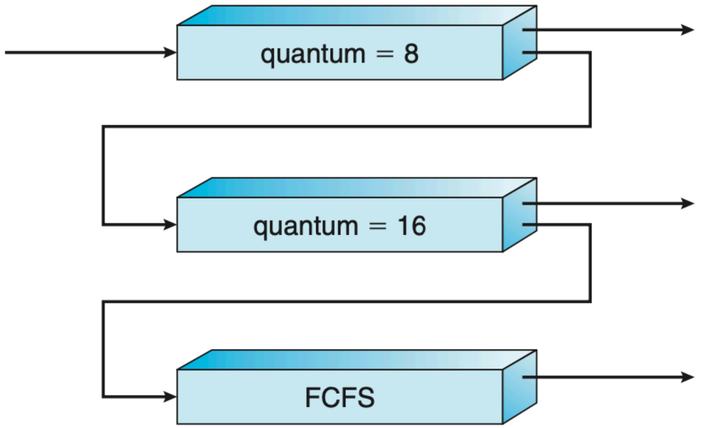


Scheduling Algorithms

- FCFS (First-Come, First-Served)
- Shortest-Job-First Scheduling
- Round-Robin Scheduling
- Priority Scheduling
- Multilevel Queue Scheduling
- **Multilevel Feedback Queue Scheduling**

Multilevel Feedback Queue Scheduling

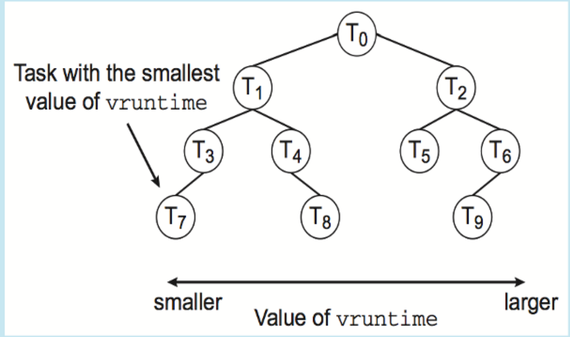
- A process can **move between the various queues**; aging can be implemented this way
- Multilevel-feedback-queue scheduler defined by the following parameters
  - number of queues
  - scheduling algorithms for each queue
  - method used to determine when to upgrade a process
  - method used to determine when to demote a process
  - method used to determine which queue a process will enter when that process needs service



Schedulers used in Linux Kernel

- O(n) scheduler
  - https://en.wikipedia.org/wiki/O(n)_scheduler
- O(1) scheduler
  - https://en.wikipedia.org/wiki/O(1)_scheduler
- Completely Fair Scheduler O(nlogn)
  - https://en.wikipedia.org/wiki/Completely_Fair_Scheduler
  - https://android.googlesource.com/kernel/common/+/experiment/kernel/sched/fair.c
- Brain Fuck Scheduler
  - https://en.wikipedia.org/wiki/Brain_Fuck_Scheduler

The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:



When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(\lg N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb_leftmost`, and thus determining which task to run next requires only retrieving the cached value.

Outline

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU Scheduling 会考
- Algorithm Evaluation

Thread Scheduling

- On most modern operating systems it is **kernel-level threads**—not processes—that are being scheduled by the operating system.

  (為爭奪有限的CPU資源發生的競爭)

- Contention Scope
  - Process Contention Scope (PCS)
    - competition for the CPU takes place among threads belonging to the same process
  - System Contention Scope (SCS)
    - competition for the CPU takes place among all threads in the system

Outline

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU Scheduling
- Algorithm Evaluation

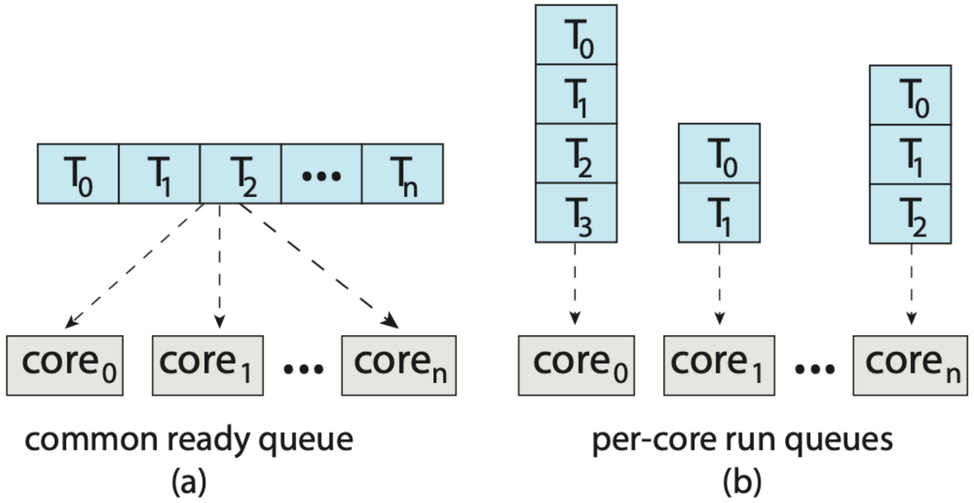
Multi-Processor Scheduling

- If multiple CPUs are available, load sharing, where multiple threads may run in parallel, becomes possible, however scheduling issues become correspondingly more complex

- Multi-Processor (Multi CPU / Multi Core)
  - Homogeneous (同質的)
    - Multicore CPUs
    - Multithreaded cores
  - Heterogeneous multiprocessing (異質的)

Approaches to Multiple-Processor Scheduling

- Asymmetric multiprocessing – only one processor accesses the system data structures, alleviating the need for data sharing (由一个能力特别强的Process来排其他所有的Process)
- **Symmetric multiprocessing (SMP)** – each processor is self-scheduling **(currently, most common)** (现今主流作法) (每个Process都自己排程自己)
  - all processes in a common ready queue
  - each processor has its own private queue of ready processes

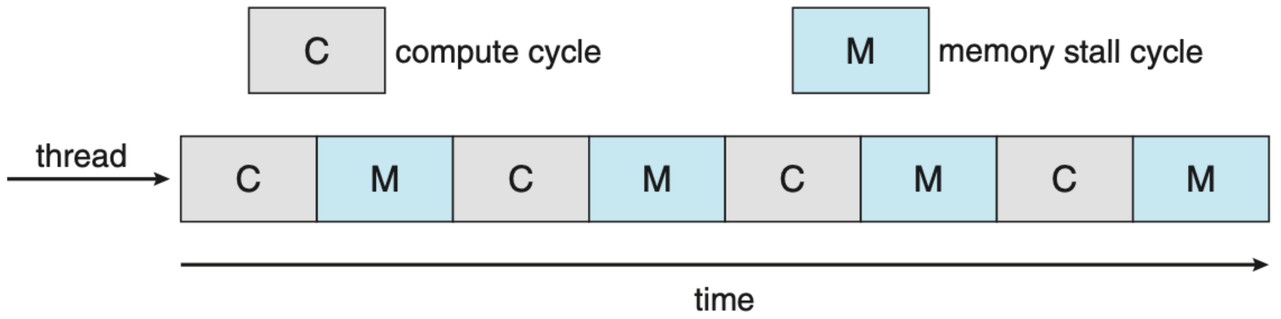


Multicore Processors

- Recent trend to place multiple processor cores on same physical chip
  - Faster and consumes less power

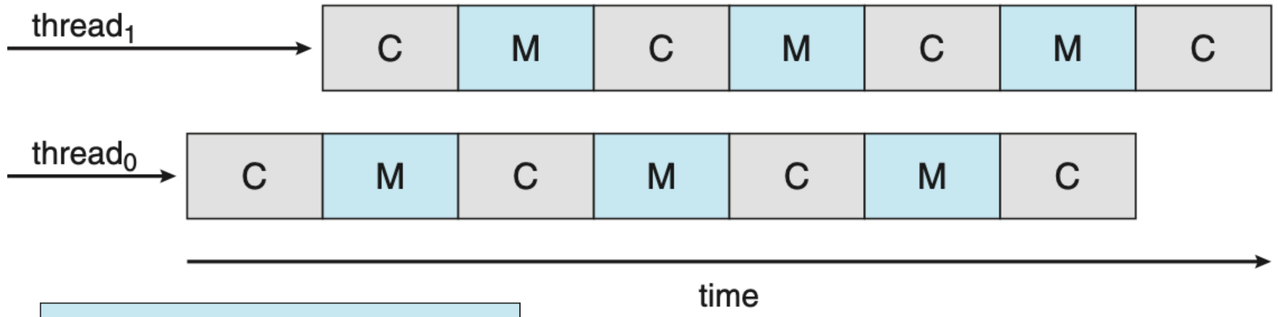
CPU太快了，所以提供了一个Hardware Thread，讓一个Process在讀mem時，另一个Process先跑

- **Memory Stall**
  - modern processors operate at much faster speeds than memory
  - cache miss

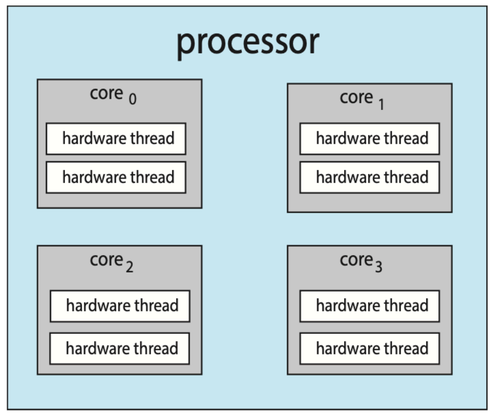


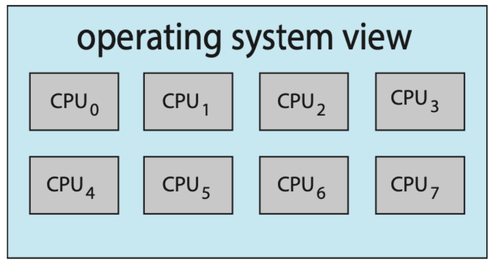
Multicore Processors

- **Hardware Threads**

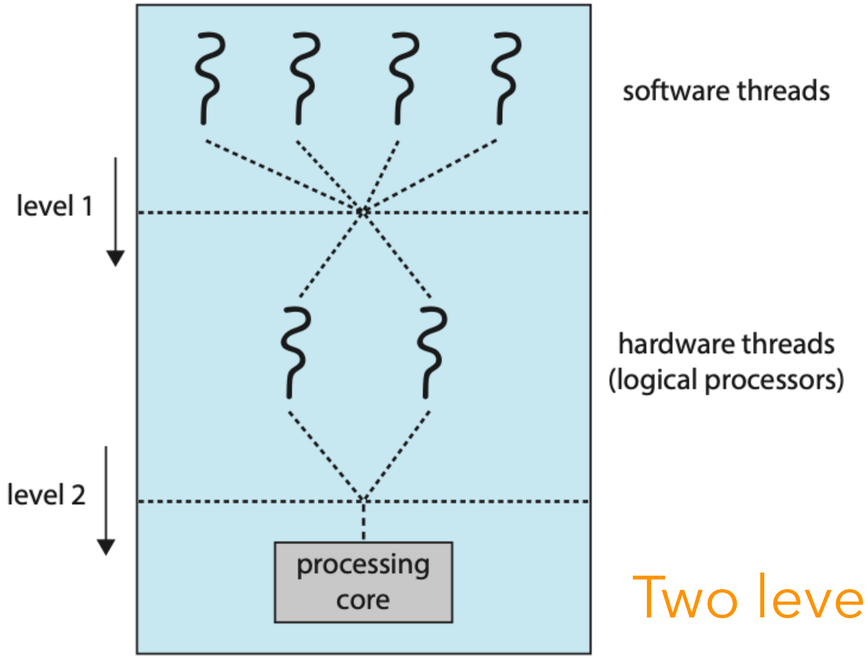


Multithreaded multicore system





Chip Threading



Two levels of scheduling

Load Balancing

負載均衡

- Keep all CPUs loaded for efficiency
- Load balancing attempts to keep workload evenly distributed
  - **Push migration** – periodic task checks load on each processor, and if found pushes task from overloaded CPU to other CPUs

    將 Process 給其他 Core
  - **Pull migration** – idle processors pulls waiting task from busy processor

    從其他Core拿 Process來

Processor Affinity

- **Processor Affinity** 親近性
  - most OSs with SMP support try to avoid migrating a thread from one processor to another and instead attempt to keep a thread running on the same processor and take advantage of a warm cache
  - Soft affinity
    - it is possible for a process to migrate between processors during load balancing
  - Hard affinity
    - allowing a process to specify a subset of processors on which it can run

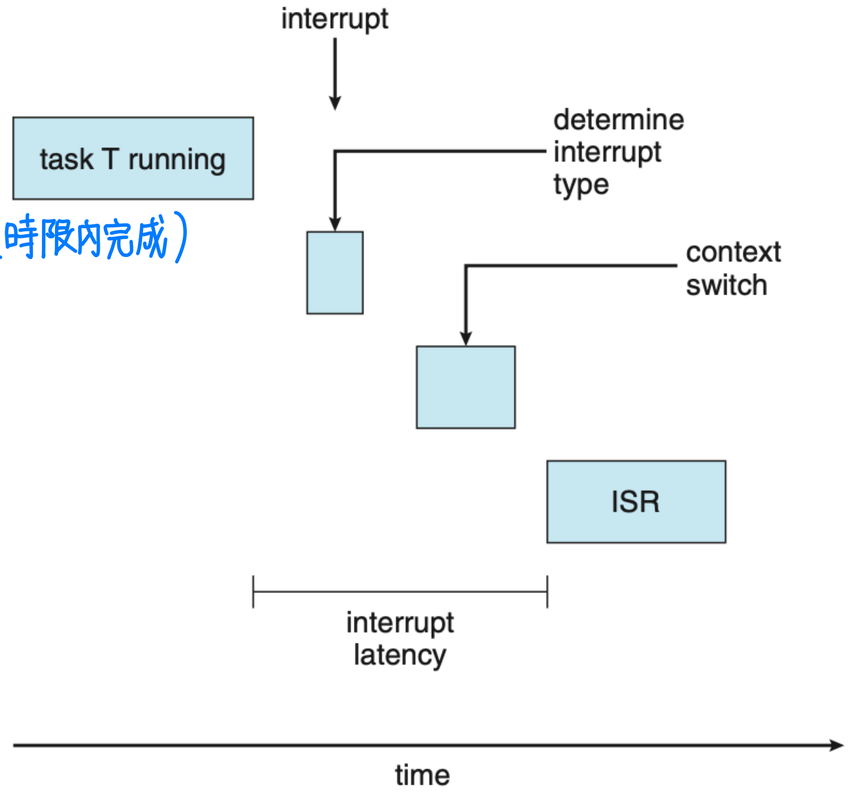
(Load Balancing 和 Affinity 要取得平衡!)

Outline

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU Scheduling 必考！
- Algorithm Evaluation

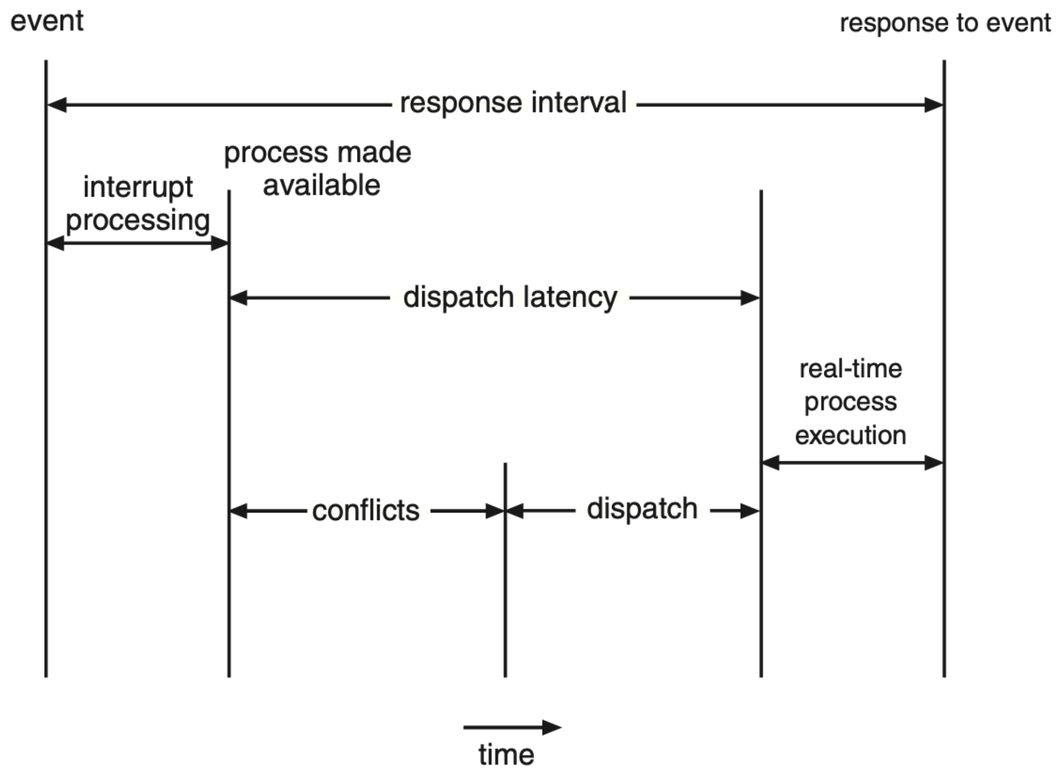
Real-Time CPU Scheduling

- Soft real-time systems
  - no guarantee as to when critical real-time process will be scheduled
- Hard real-time systems
  - task must be serviced by its deadline (所有Process必須在規定時限內完成)
- Two types of latencies affect performance
  - **Interrupt latency** – time from arrival of interrupt to start of routine that services interrupt (中斷延遲)
  - **Dispatch latency** – time for schedule to take current process off CPU and switch to another (= context switch)



Real-Time CPU Scheduling

- Conflict phase of dispatch latency:
  - Preemption of any process running in kernel mode
  - Release by low-priority process of resources needed by high-priority processes



Priority-based Scheduling

会考

- For real-time scheduling, scheduler must support preemptive, priority-based scheduling
  - But only guarantees soft real-time
- For hard real-time must also provide ability to meet deadlines
  - The processes are considered periodic
    - Has processing time t, deadline d, period p

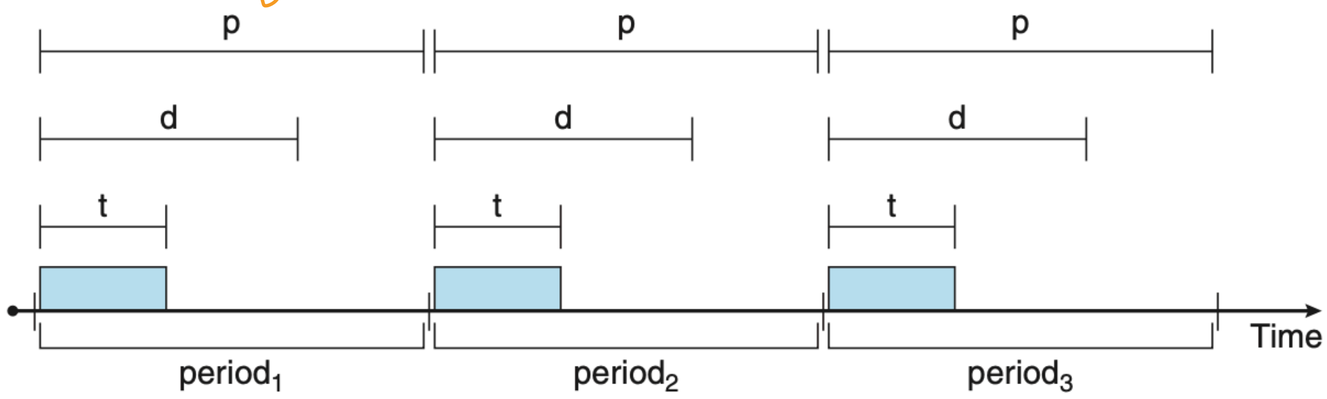
通常來說，t一定必<d

process的執行時間

process的死線

周期(e.g.有些task需要定期去執行)

    - $0 \leq t \leq d \leq p$
    - Rate of periodic task is 1/p



Rate-Monotonic Scheduling

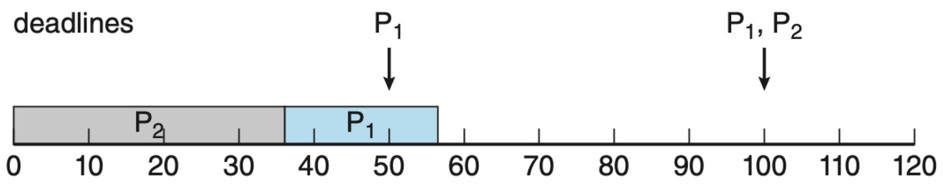
*会考，必考 (單调的)

- A priority is assigned based on the inverse of its period
  - Example (p小的先做)
    - $P_1: p_1 = 50, t_1 = 20$
    - $P_2: p_2 = 100, t_2 = 35$

> Rate = $\frac{1}{p}$, obviously, P_1's rate higer than P_2's rate → Rate-based Scheduling

    - The deadline for each process requires that it complete its CPU burst by the start of its next period (p=d, 周期=死線)

*会考畫这个圖!

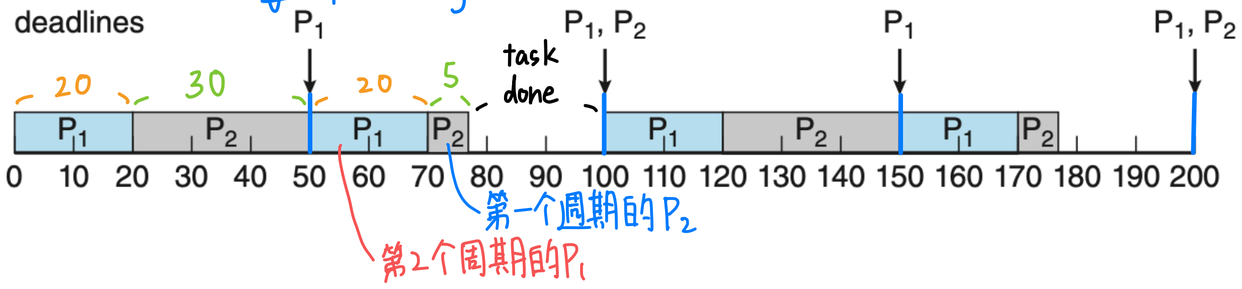


Priority-Based Scheduling

P_2 has a higher priority than P_1

若P2 Priority 較高，会讓P1 趕不上死線!

*P_1 Rate Higher then P_2, so switch



Rate-Monotonic Scheduling

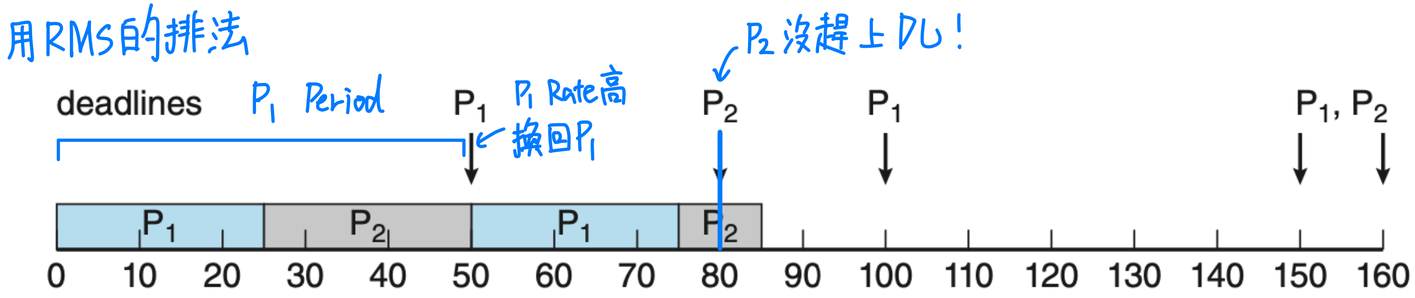
Earliest-Deadline-First Scheduling

*会考、必考　死線先到先排

- Priorities are assigned according to **deadlines**
  - EDF scheduling does not require that processes be periodic
  - Example
    - $P_1: p_1 = 50, t_1 = 25$
    - $P_2: p_2 = 80, t_2 = 35$

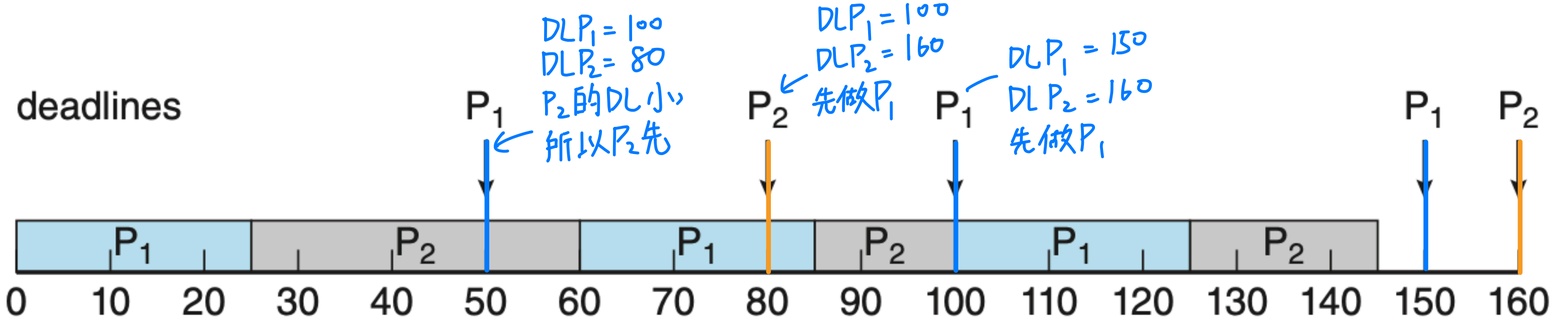
EDF沒有固定Priority!

用RMS的排法



Missing deadlines with rate-monotonic scheduling

會考畫這個圖很重要!



Earliest-deadline-first scheduling

Schedulable

- Liu and Layland's Utilization Bound

The Liu & Layland utilization bound is a mathematical condition used to determine whether a set of periodic real-time tasks is schedulable under the Rate-Monotonic Scheduling (RMS) algorithm.

It provides a sufficient (but not necessary) condition for schedulability.

—

📘 Definition:

For a set of n independent periodic tasks, each task Ti has:

- Ci = computation (execution) time (t)
- Ti = period (assumed equal to the deadline) (d)

The total CPU utilization is defined as:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

小於1有機会成功

According to Liu & Layland, the task set is guaranteed to be schedulable under RMS if:

$$U \leq n \left(2^{1/n} - 1\right)$$

This inequality gives a utilization threshold that depends on the number of tasks n.

Schedulable

🧱 EDF Schedulability Test (Utilization-Based)

For a set of n independent periodic tasks where:

- Each task Ti has execution time Ci and period Ti
- The relative deadline of each task is equal to its period (Di = Ti)

Then the system is schedulable under EDF if:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1$$

EDF≤1，必成功！

—

✅ Interpretation:

- If total CPU utilization U is less than or equal to 1, all tasks are guaranteed to meet their deadlines under EDF.
- If U > 1, the system is overloaded, and at least one task will miss its deadline.

Outline

- Basic Concepts
- Scheduling Criteria
- Scheduling Algorithms
- Thread Scheduling
- Multi-Processor Scheduling
- Real-Time CPU Scheduling
- Algorithm Evaluation 評估 演算法

Algorithm Evaluation

有可能会考

- Determine criteria, then evaluate algorithms
  決定 Criteria (ppt. P12)　再來評估一个演算法的好壞
- Evaluation methods
  - Deterministic Modeling
  - Queueing Models
  - Simulations
  - Implementation

Deterministic Modeling

- One type of analytic evaluation
- Takes a particular predetermined workload and defines the performance of each algorithm for that workload

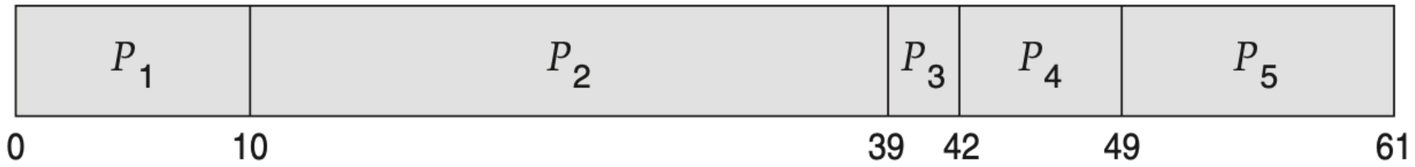
給定一个事先决定好的资料来評估

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

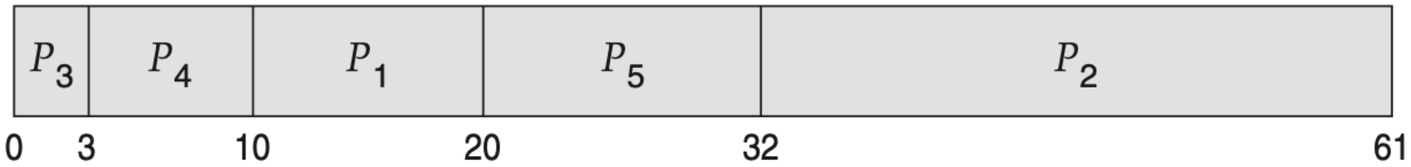
particular predetermined workload

Deterministic Modeling

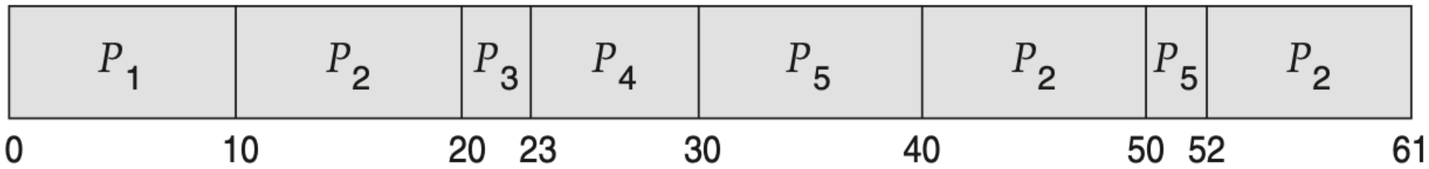
- For each algorithm, calculate minimum average waiting time
- Simple and fast, but requires exact numbers for input, applies only to those inputs



FCFS
(28 milliseconds)



Nonpreemptive SJF (Best)
(13 milliseconds)



Round Robin
(23 milliseconds)

Queueing Models

- Describes the arrival of processes, and CPU and I/O bursts probabilistically (distribution) 用數學式來描述process的arrival
  - Commonly exponential (memoryless distribution), and described by mean
  - Computes average throughput, utilization, waiting time, etc
- Computer system described as network of servers, each with queue of waiting processes
  - Knowing arrival rates and service rates
  - Computes utilization, average queue length, average wait time, etc

Little's Formula

利特爾法則

- n = average queue length
- W = average waiting time in queue
- λ = average arrival rate into queue
- Little's law – in steady state, processes leaving queue must equal processes arriving, thus:

  $$n = \lambda \times W$$

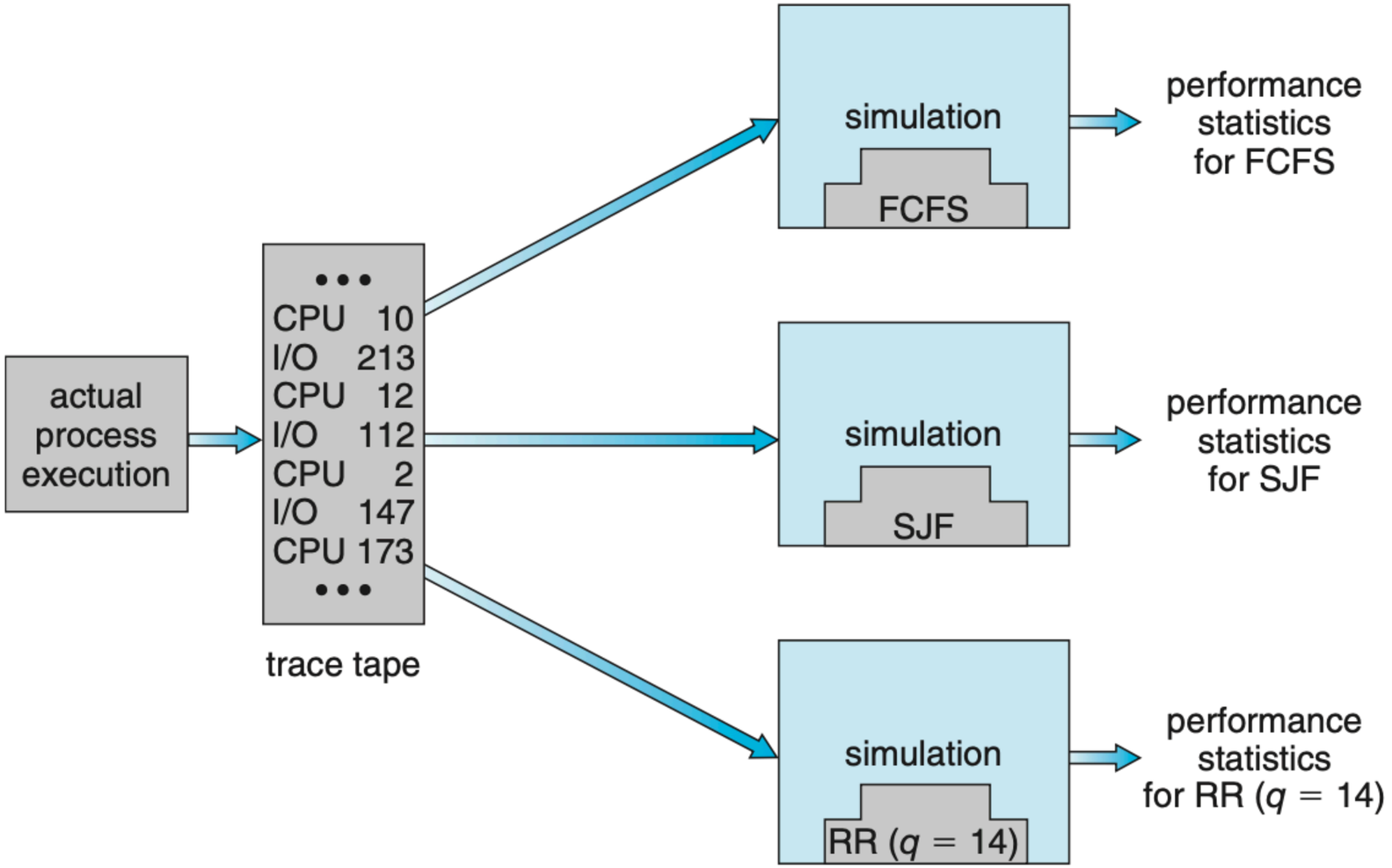
  - Valid for any scheduling algorithm and arrival distribution
- For example, if on average 7 processes arrive per second, and normally 14 processes in queue, then average wait time per process = 2 seconds

Simulations

用模擬器來直接評估一个演算法的好壞

- The simulator has a variable representing a clock. As this variable's value is increased, the simulator modifies the system state to reflect the activities of the devices, the processes, and the scheduler. As the simulation executes, statistics that indicate algorithm performance are gathered and printed.
- Event-driven Simulator - Network Simulator 3
- Data to drive simulation gathered via
  - Random number generator according to probabilities
  - Distributions defined mathematically or empirically
  - Trace tapes record sequences of real events in real systems

Simulations



Implementation

- Even simulations have limited accuracy
- Just implement new scheduler and test in real systems
  - High cost, high risk
  - Environments vary

References

- https://scheduling-algorithm-simulator.vercel.app/
- https://process-scheduling-solver.boonsuen.com/
- https://cpu-scheduling-sim.netlify.app/
- https://mukul2310.github.io/cpu-scheduler-visualiser/
- https://aadhil2k4.github.io/Process_Scheduling_Calculator/